1:00 PM Thu 27 Feb

All tools Edit Export E-Sign

Computer Organization and Architecture
(Code: CS256)

Lecture-17: ALU Operations

Course Faculty

Dr. B. R. Bhowmik

Assistant Professor

Dept. of CSE

Website: https://sites.google.com/view/biswajitrbhowmik
https://sites.google.com/view/vicayana-gaveshana

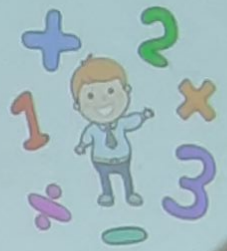
Course Email: sangsangstha@gmail.com

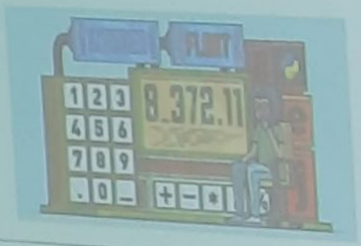
1:01 PM Thu 27 Feb

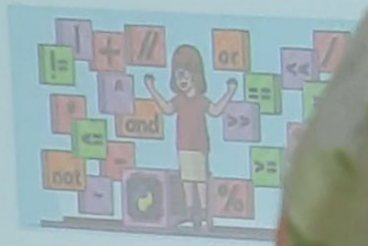
All tools Edit Export E-Sign

Outlines

- Arithmetic Operations
  - Addition
  - Subtraction
  - Multiplication
  - Division
- Logical Operations
- Shift Operations







1:02 PM Thu 27 Feb

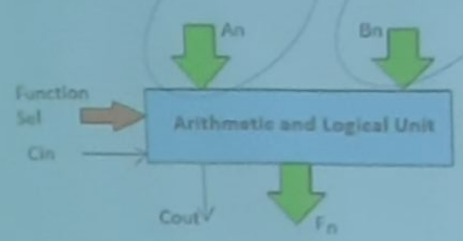
All tools Edit Export E-Sign

Learning Outcomes

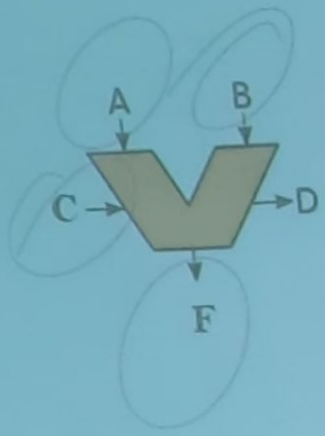
- Demonstrate and justify standard for addition, subtraction, multiplication and division of whole numbers, integers, fractions, and decimals.
- Design of digital circuits as a part of ALU components for these operations.
- Handling floating point operations over ALU.
- Accomplishing multiple logical and shift operations.

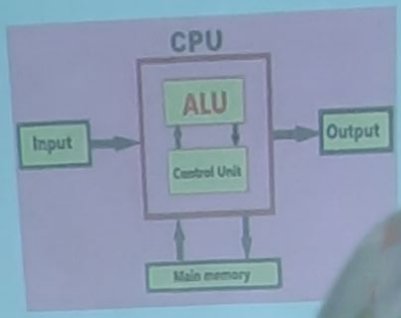
Arithmetic Logic Unit

- It has 2 input vectors:
  - These may be called the A- and B-vector or the R- and S-vector
  - The inputs are as wide as the registers/system bus (e.g., 16, 32 bits)
- It has 1 output vector
  - Usually denoted F
- ALU is basically a black-box



Block Diagram for ALU





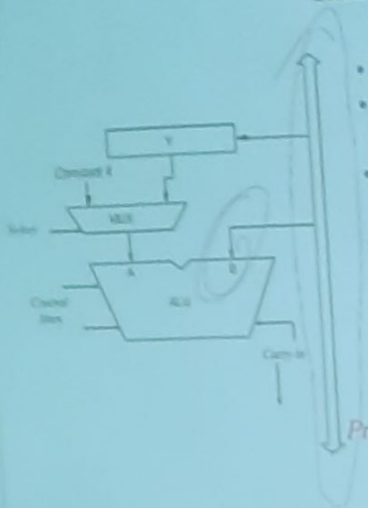
ALU Operations

- Add (A+B)
- Add with Carry (A+B+Cin)
- Subtract (A-B)
- Subtract with Borrow (A-B-Cin)
- Subract reverse (B-A)
- Subract reverse with Borrow (B-A-Cin)
- Negative A (-A)
- Negative B (-B)
- Increment A (A+1)
- Increment B (B+1)
- Decrement A (A-1)
- Decrement B (B-1)
- Multiplication
- Division

- Logical AND
- Logical OR
- Logical XOR
- Not A
- Not B
- Masking
- Conditional AND/OR (uses Mask)
- Shift

Edit Export E-Sign

ALU Connections



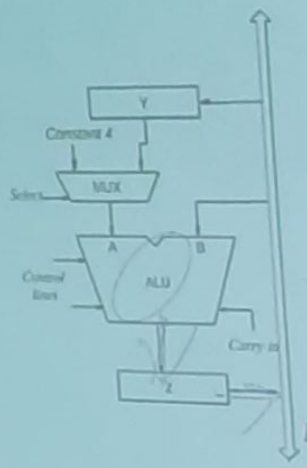
- ALU must have only one input connection from the bus.
- The other input must be stored in a holding register called Y register.
- A multiplexer selects among register Y and 4 depending upon select line.
- One operand of a two-operand instruction must be placed into the Y register before the other operand must be placed onto the bus.

ALU connections to the bus

Thu 27 Feb

All tools Edit Export E-Sign

ALU Connections



- *Identical reasoning tells us that there must be an output register Z which collects the output of the ALU at the end of each cycle.*
- *This way, there can be*
  --one operand in theY register
  --one operand ON THE BUS
  --the result stored in the Z register

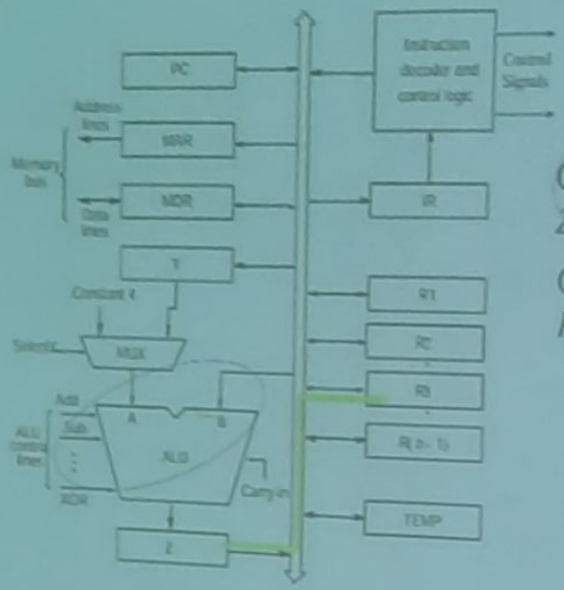
Performing An Arithmetic Operation

Add the contents of registers *R1* and *R2* and place the result in *R3*.

$$R3 = R1 + R2$$

- Place the contents of register *R1 into the* Y register in the first clock cycle.
- Place the contents of register *R2 onto the bus* in the second clock cycle.
- Both inputs to the ALU are now valid.
- Select register Y, and assert the ALU command F=A+B.
- In the third clock cycle, Z register has latched the output of the ALU.
- Thus the contents of *the Z* register can be copied into *register R3*.

Performing Addition Operation



Clock cycle 3:
Z_{out}, $R3_{in}$

Clock cycle 4:
R3 has the sum.

Performing Addition Operation

Clock Cycle 1: $R1_{out}, Y_{in}$ $(Y=R1)$

Clock Cycle 2: $R2_{out}, SelectY, Add, Z_{in}$ $(Z = R1+R2)$

Clock Cycle 3: $Z_{out}, R3_{in}$ $(R3=Z)$

Performing Addition Operation

Events as seen by the system registers, bus, ALU over time.

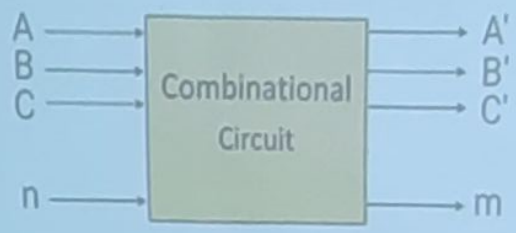
cycle:	controls active	what the bus sees	what Y has	output of ALU
start of 1	$R1_{out}$, Y_{in}	contents of R1	–unknown	–unknown
end of 1 – start of 2	$R1_{out}$, Y_{in}	contents of R1	R1	–unknown
end of 2 – start of 3	$R2_{out}$, F=A+B	contents of R2	R1	F=A+B=R1+R2 but this is not valid yet
end of 3 latches bus	$R2_{out}$, F=A+B, Z_{in}	contents of R2	R1	F=A+B=R1+R2 (now valid)
	Z_{out}, $R3_{in}$	contents of Z	R1	–unknown
	Z_{out}, $R3_{in}$	contents of Z	R1	–unknown (but R3 contents)

ALU Operations

- $RC = RA\ op\ RB$
- Clock cycle 1:
  - Move RA to Y register.
  - RA_{out}, Y_{in}
- Clock cycle 2:
  - Put RB on the bus, perform $F = RA\ op\ RB$, and transfer the result to Z.
  - RB_{out}, $(RA\ op\ RB)=1$, $Select Y$, Z_{in}
- Clock cycle 3:
  - Put Z on the bus, and load Z into RC.
  - Z_{out}, RC_{in}

Adder for Addition

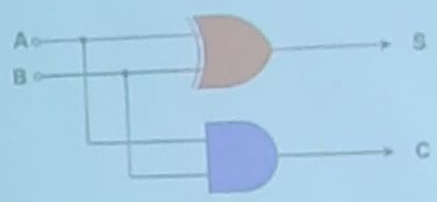
- Half Adder → 2 bit addition
- Full Adder → 3 bit addition
- N-Bit Adder → N>3



Half Adder

- Components Required

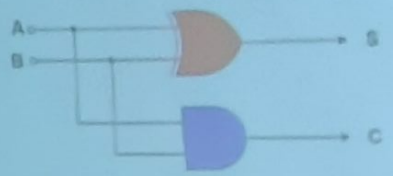
SI.No.	COMPONENT	SPECIFICATION	QTY.
1.	AND GATE	IC 7408	1
2.	X-OR GATE	IC 7486	1
3.	NOT GATE	IC 7404	1
4.	OR GATE	IC 7432	1



Half Adder

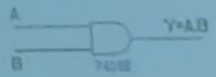
- Components Required

SL No.	COMPONENT	SPECIFICATION	QTY.
1.	AND GATE	IC 7408	1
2.	X-OR GATE	IC 7486	1
3.	NOT GATE	IC 7404	1
4.	OR GATE	IC 7432	1



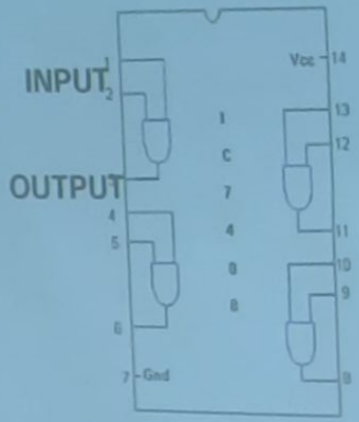
PIN DIAGRAM for Half Adder

AND GATE (IC 7408):

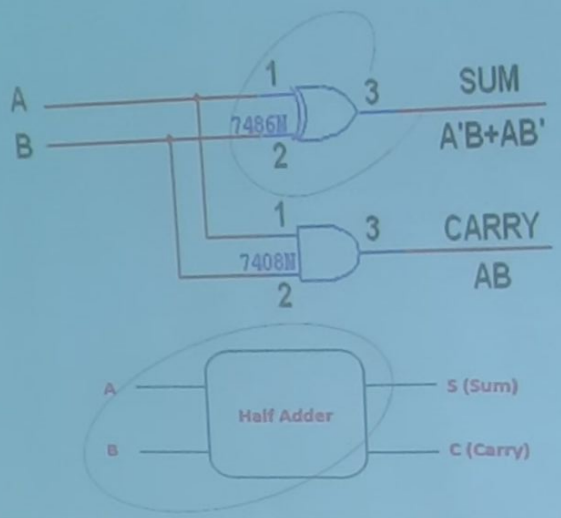


TRUTH TABLE

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1



Circuit Diagram : Half Adder



Application of Half Adder

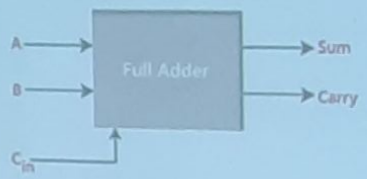
- The ALU (arithmetic logic circuitry) of a computer uses half adder to compute the binary addition operation on two bits.
- Half adder is used to make full adder as a full adder requires 3 inputs, the third input being an input carry, i.e.,
- We will be able to cascade the carry bit from one adder to the other.

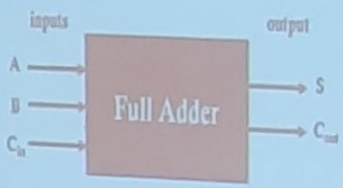
Full Adder

A

TRUTH TABLE:

A	B	C	Sum (S)	Carry (C)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





Design of Full Adder

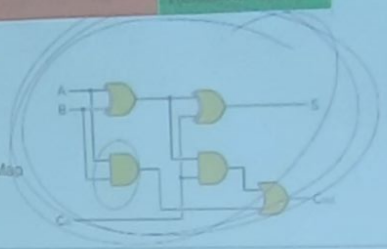
A	B	C	Sum (S)	Carry (C)
0	0	0	0	0
				0
				0
				1 (midterm = A'BC)
0	1	1	0	
				0
				1 (midterm = AB'C)
1	0	1	0	1 (midterm = ABC')
1	1	0	0	1 (midterm = ABC)

Boolean Equation:

SUM : S= A'B'C+A'B C'+AB'C'+ABC (EX-OR GATE)

CARRY: C=A'BC+AB'C+ABC'+ABC Reducing by K-Map

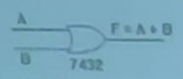
C=AB+BC+CA



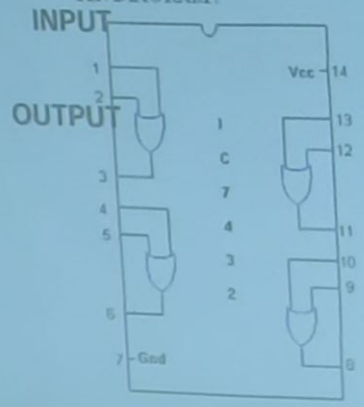
Pin Diagram of Full Adder

OR GATE:

SYMBOL :



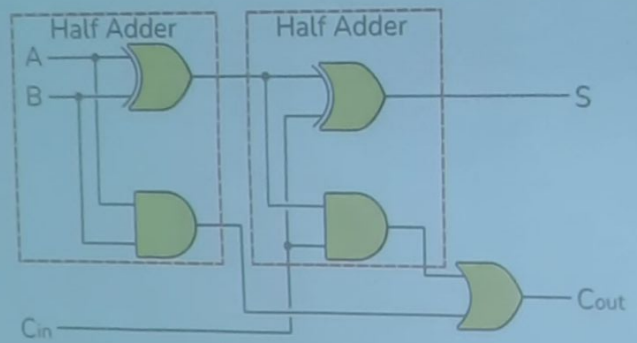
PIN DIAGRAM :



TRUTH TABLE

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

Full Adder by Half Adders

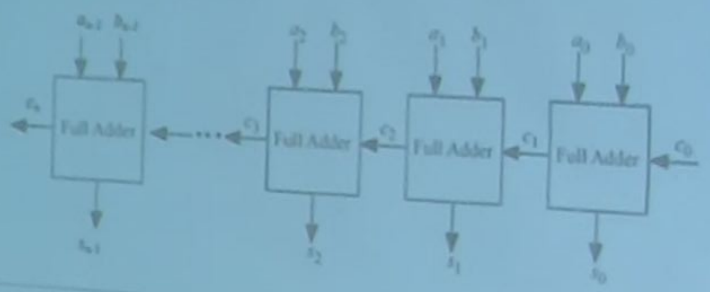


Application of Full Adder

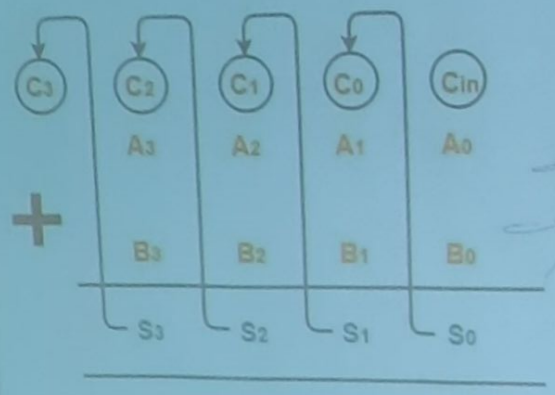
- Full adders are used in calculators.
- Full adders also help in carrying out multiplication of binary numbers.
- Full adders are also used to realize critic digital circuits like multiplexers.
- Full adders are used to generate memory addresses.
- Full adders are also used in generation of program counterpoints.
- Full adders are also used in GPU (Graphical Processing Unit).

N-bit Adder

- Adders for arbitrarily large binary numbers can be constructed by cascading full adders.
- *The carry bit "ripples" from one stage to the next.*
- The n-bit adder built from n one-bit full adders.
- This kind of adder is called a *ripple carry adder*, since each carry bit "ripples" to the next full adder.
- Each full adder inputs a Cin, which is the Cout of the preceding adder.



Ripple Carry Adder

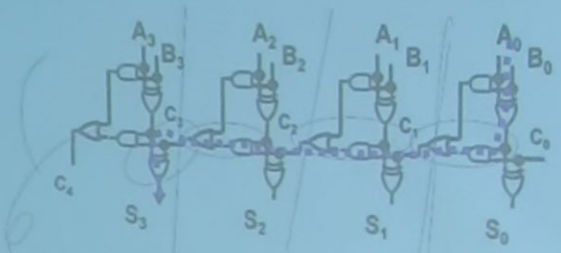


Description	Subscript 3 2 1 0	Name
Carry In	0 1 1 0	C_i
Augend	1 0 1 1	A_i
Addend	0 0 1 1	B_i
Sum	1 1 1 0	S_i
Carry out	0 0 1 1	C_{i+1}

Adding two 4-bit Numbers

Limitations of n-bit Adder

- A drawback of an n-bit adder is that the carry information has to propagate through all stages.
- For example, there is an electrical path from A0 and B0 to C3 in the circuit.
- This path traverses 2N-1 gates in an N-bit adder.
- This may lead to undesirably long delays before the output stabilizes.
- The problem can be overcome by more complex carry computations.

N

2N-1